

The Universe as Computation and Hardware: The Nexus Recursive Framework

Driven By Dean A. Kulik

June 2026

Executive summary

The user's Nexus primitives map surprisingly well onto established computational and hardware primitives when interpreted as constraint systems rather than object systems. The strongest correspondences are these: **fold/reflection** maps to logical complement and physical inversion, especially the CMOS inverter; **rails/gaps** map to supply rails, differential pairs, and sense-amplified bitlines where meaning lives in a constrained difference rather than an absolute value; and **collapse** maps to regenerative settling in latches, flip-flops, sense amplifiers, attractor memories, and modern Hopfield updates. In that sense, the Nexus claim that "reality lives in the gap" has a direct hardware analogue: digital logic, SRAM/DRAM readout, and differential photonic and spintronic memories all compute by enforcing constraints on opposing states and then regenerating a small gap into a full decision. [1]

The proposed **edge-first hex lattice** is mathematically coherent. If each hex cell has six edge bits $e_0, \dots, e_5$ with opposite-edge complement constraints

$$e_{i+3} = \neg e_i \quad (i = 0,1,2),$$

then each cell has only three independent edge bits, hence exactly **eight legal local motifs**. When shared-edge consistency is imposed across an open $L_x \times L_y$ hex patch, the global degrees of freedom migrate from area to boundary: the entire interior can be parameterized by three alternating rail families, and the free-bit count scales as

$$L_y + (L_x + L_y - 1) + L_x = 2(L_x + L_y) - 1,$$

which is perimeter-like rather than area-like. That is the cleanest formal expression of the user’s intuition that “the edges matter” and that cells are closures of rail tension rather than primary state holders.

Between clock ticks, the right physical model is not synchronous Life-like rewriting but **continuous-time propagation plus settling**: interconnect delay, charge sharing, regeneration, and metastable dwell occur before any clean binary readout exists. The result is an **emergent coherence ceiling**: on a graph CA it appears as finite-speed locality; in hardware it appears as the maximum clock frequency or handshake rate that still allows all local collapses to settle before observation. The engineering version of “genlock” is therefore not an arbitrary global ruler but the unavoidable bound imposed by propagation delay, RC interconnect, regenerative resolution time, setup/hold margins, and skew/jitter. [2]

The user’s “memory-as-collapse” intuition already has strong literature support. Classical Hopfield networks retrieve from partial cues by descending an energy function into attractor basins; modern Hopfield or dense associative memories greatly increase capacity; and Ramsauer et al. explicitly show that the modern Hopfield update is mathematically equivalent to transformer attention. The user-provided **Memory by Collapse** document then adds a Nexus-specific engineering proposal and measurements: one-shot local writing, recall by settling from fragments, no forgetting in a toy stream after 1,000 one-shot writes, and an energy ledger that sharply disfavors global gradient-based rewriting under the tested setup. That does not prove a universal alternative to transformers, but it does justify a serious architectural program around **hierarchies of collapse cells** rather than single giant forward passes. [3]

Nexus primitives as computational and hardware primitives

The most productive way to formalize Nexus is to treat each primitive as a **constraint-bearing operator** with a computational reading and a hardware embodiment. In that view, the abstract language of echo, subdivision, rails, gaps, fold, and genlock becomes technical rather than metaphorical.

Nexus to hardware mapping

Nexus primitive	Formal reading	Computational primitive	Hardware realization	Nexus interpretation
Whole / frame	admissible state space	normalization, Boolean domain, conserved interpretation space	V_{DD}/GND , precharge reference, matched differential pair, allowed phase window	the field that makes states readable
Echo	identity restoration, attractor return	content-addressable recall, state retention, idempotent restoration	latch hold state, SRAM $Q/\overline{Q}$, recurrent attractor basin	class / same-again
Subdivision	address, allocation, decomposition	indexing, fan-out, routing, softmax weight distribution	wordlines, bitlines, buses, memory rows, repeated instances	many-from-one
Fold / reflection	complement or inverse relation	NOT, inversion, phase flip	CMOS inverter, differential inversion, MZI phase inversion	subtraction through a seam

Nexus primitive	Formal reading	Computational primitive	Hardware realization	Nexus interpretation
Seam	unstable midpoint / switching threshold	decision boundary, comparator threshold	inverter switching point, latch metastable saddle, precharge midpoint	the gap that becomes readable
Rails	opposing constraints	dual-rail encoding, differential signaling, legal endpoints	supply rails, BL/BLB, true/complement wires, R_P/R_{AP} in MTJ	constraint lines
Gap	admissible difference between rails	state variable, signal, match/mismatch, delta	$\Delta V, \Delta I, \Delta R$, phase difference, charge differential	lived state between constraints
Collapse / settling	convergence under constraints	attractor descent, arbitration, completion detection	sense amplifier regeneration, cross-coupled latches, Ising latch arrays, CAM match resolution	many-to-one closure
Class / echo multiplicity	one invariant rule, many instances	standard-cell class, many instantiations	one inverter or bitcell layout placed millions of times	one class, many echoes
Bus	parallel transport of constrained gaps	vector state transfer	wire bundles, differential buses, optical waveguide arrays	many gaps in motion
Clock / genlock	coherence fence for observation	sample window, phase boundary	system clock, ready/ack handshake window	propagation ceiling made explicit

The gate-level part of this mapping is especially sharp. The user’s “subtractive” reading is not merely philosophical: in binary logic, the inverter really is subtraction from the frame,

$$\text{NOT}(x) = 1 - x,$$

and in bipolar logic $s \in \{-1, +1\}$, complement is simply

$$\text{NOT}(s) = -s.$$

Once inversion exists, NAND and NOR become “fit then subtract” operators, and XOR becomes a mismatch detector:

$$\text{NAND}(a, b) = 1 - ab, \quad \text{NOR}(a, b) = 1 - (a \vee b), \quad \text{XOR}(a, b) = a \oplus b = a + b - 2ab.$$

That is why a Nexus reading of logic naturally centers NOT first, then universal inverting gates next, and only then larger compositions. In modern memory arrays this is not just theory: SRAM-based in-memory computing has demonstrated NAND, NOR, XOR, and related Boolean operations directly in differential

memory structures, and photonic SRAM proposals now embed XOR inside differential memory cells themselves. [4]

Gates, latches, buses, and clocks in Nexus form

Primitive	Formal rule	Canonical hardware	Nexus reading
NOT	$1 - x$	CMOS inverter	reflection / subtraction
NAND	$\neg(a \wedge b)$	static CMOS NAND	fit, then subtract from whole
NOR	$\neg(a \vee b)$	static CMOS NOR	occupancy, then subtract
XOR	$a \oplus b$	pass-transistor, differential, or in-memory XOR	seam detector / difference detector
Latch	positive feedback over two complements	cross-coupled inverters	collapse cell with two basins
Flip-flop	latch + time gate	master-slave edge-triggered storage	sampled collapse
Bus	vector of constrained channels	multi-wire bundle, often dual-rail or differential	many moving gaps
Clock	observation fence	periodic or handshake boundary	coherence phase line

The step from gate to memory is the key conceptual turn. A latch is not “a gate with memory” in Nexus language; it is **the first physical collapse cell**. Two cross-coupled inverters create a bistable landscape with two attractors and one unstable midpoint. That same regenerative motif reappears in SRAM bitcells, sense amplifiers, arbiters, dynamic comparators, and even latch-based Ising machines, where cross-coupled bistable elements are treated as physically settling computational units. [5]

The user’s “one class, many echoes” intuition also maps cleanly onto integrated-circuit design practice. A standard-cell library defines one inverter class, one NAND class, one latch class, and so on; place-and-route instantiates those classes many times. SRAM and photonic SRAM go further: a single cross-coupled bitcell class is tiled into very large arrays, with each cell acting as an echo or instance of the same invariant constraint pattern. [6]

Rails as constraints and gaps as state

Nexus becomes most physical when “rails” are taken literally. In CMOS, the rails are V_{DD} and ground. In SRAM, the cell stores complementary states Q and $\bar{Q}$. In DRAM, bitlines are precharged to a midpoint and a tiny charge-sharing differential is then regenerated. In asynchronous delay-insensitive design, one bit is often represented by a pair of wires in a 1-of-2 dual-rail code under a return-to-zero protocol. In spintronics, the meaningful state is a resistance difference between magnetic configurations. In coherent photonics, the state is carried by phase or amplitude difference between arms of an interferometer or by a resonance shift relative to a reference state. In all of these, the visible datum is a **gap variable** subject to rail constraints. [7]

Edge-first hex lattice model

The user asked for a Game-of-Life inversion in which **edges are primary** and each hex cell's opposite edges are reflections of one another. That model can be written exactly.

Local state and legality

Let each hex cell c have six ordered edge states

$$e_c = (e_0, e_1, e_2, e_3, e_4, e_5), \quad e_i \in \{0,1\}.$$

Impose the opposite-edge NOT constraints

$$e_3 = \neg e_0, \quad e_4 = \neg e_1, \quad e_5 = \neg e_2.$$

Equivalently,

$$e_i \oplus e_{i+3} = 1 \quad (i = 0,1,2).$$

So only three edge bits are independent. Each legal hex cell therefore has one of exactly **eight** local motifs.

Legal hex motifs

In bipolar form, let the independent axes be $A, B, C \in \{-1, +1\}$. Then every legal cell is

$$(A, B, C, -A, -B, -C).$$

In binary form $1 \leftrightarrow +1, 0 \leftrightarrow -1$, the motifs are:

Motif ID	Seed (A, B, C)	Full edge string $(e_0 e_1 e_2 e_3 e_4 e_5)$	Note
M0	$(-, -, -)$	000111	balanced, three 0s / three 1s
M1	$(-, -, +)$	001110	balanced
M2	$(-, +, -)$	010101	balanced
M3	$(-, +, +)$	011100	balanced
M4	$(+, -, -)$	100011	balanced
M5	$(+, -, +)$	101010	balanced
M6	$(+, +, -)$	110001	balanced
M7	$(+, +, +)$	111000	balanced

A first important consequence falls out immediately: **every legal cell is locally neutral**. Because each opposite pair contributes one 1 and one 0, every motif has Hamming weight three. The cell is therefore not a carrier of arbitrary six-bit content. It is a **balanced closure pocket**.

Shared-edge consistency

Now impose the second rule: shared edges must be identical for neighboring cells. If a cell c and its neighbor $n_i(c)$ share edge i , then

$$e_{c,i} = e_{n_i(c),i+3 \bmod 6}.$$

This is the hardware-style interface rule: one physical edge cannot hold two different values.

Axial parameterization

Choose axial coordinates (q, r) for the hex lattice and let directions 0,1,2 correspond to the three independent edge orientations. Define

$$A_{q,r} = e_{(q,r),0}, \quad B_{q,r} = e_{(q,r),1}, \quad C_{q,r} = e_{(q,r),2}.$$

Shared-edge consistency plus opposite-edge NOT then gives three simple line constraints:

$$A_{q+1,r} = \neg A_{q,r},$$

$$B_{q+1,r-1} = \neg B_{q,r},$$

$$C_{q,r-1} = \neg C_{q,r}.$$

In bipolar form these become alternating-sign relations,

$$A_{q+1,r} = -A_{q,r}, \quad B_{q+1,r-1} = -B_{q,r}, \quad C_{q,r-1} = -C_{q,r}.$$

So the entire legal lattice can be parameterized by three families of boundary seeds:

$$A_{q,r} = (-1)^q a_r,$$

$$B_{q,r} = (-1)^q b_{q+r},$$

$$C_{q,r} = (-1)^r c_q,$$

where $a_r, b_{q+r}, c_q \in \{-1, +1\}$.

This is the decisive result. On an open $L_x \times L_y$ rhombic patch, the free degrees of freedom are:

- L_y bits for the a_r family,
- $L_x + L_y - 1$ bits for the diagonal b_{q+r} family,
- L_x bits for the c_q family.

Therefore:

$$N_{\text{free}} = L_y + (L_x + L_y - 1) + L_x = 2(L_x + L_y) - 1.$$

The interior is not free. It is **compiled from the perimeter**. That is the rigorous version of “the grid falls into place.”

Three rails and two gaps

The user’s rail image also has a clean local encoding. On any ordered triple of adjacent parallel rails (r_0, r_1, r_2) , define the two gap variables

$$g_1 = r_0 \oplus r_1, \quad g_2 = r_1 \oplus r_2.$$

Now three rails create two admissible gaps, both sharing the middle rail. In the hex model, each orientation family A , B , or C acts as a rail family, while the visible local state of a hex cell is the closure of three such alternation constraints. The “object” is therefore not the rail itself; it is the **gap pattern permitted by the rail arrangement**.

Transition rules

There are two useful update regimes for this automaton.

Binary synchronous rule

For an idealized CA sampled at integer ticks, the update can be written as a thresholded consistency rule on the three independent edge families:

$$\begin{aligned} A_{q,r}[n+1] &= \text{sgn}(h_{q,r}^A[n] - A_{q-1,r}[n] - A_{q+1,r}[n]), \\ B_{q,r}[n+1] &= \text{sgn}(h_{q,r}^B[n] - B_{q-1,r+1}[n] - B_{q+1,r-1}[n]), \\ C_{q,r}[n+1] &= \text{sgn}(h_{q,r}^C[n] - C_{q,r-1}[n] - C_{q,r+1}[n]), \end{aligned}$$

with $\text{sgn}(0)$ either held over or treated as metastable. The h terms are optional boundary clamps, memory cues, or injected drives.

Continuous settling rule

For hardware-faithful simulation, a better version is gradient descent on a local consistency energy:

$$E = \frac{J_A}{2} \sum_{q,r} (A_{q,r} + A_{q+1,r})^2 + \frac{J_B}{2} \sum_{q,r} (B_{q,r} + B_{q+1,r-1})^2 + \frac{J_C}{2} \sum_{q,r} (C_{q,r} + C_{q,r-1})^2 - \sum_{q,r} (h^A A + h^B B + h^C C).$$

Then evolve

$$\tau \dot{A}_{q,r} = -\frac{\partial E}{\partial A_{q,r}}, \quad \tau \dot{B}_{q,r} = -\frac{\partial E}{\partial B_{q,r}}, \quad \tau \dot{C}_{q,r} = -\frac{\partial E}{\partial C_{q,r}},$$

optionally with saturation $x \mapsto \tanh(\beta x)$ and noise $\eta(t)$. Binary observation happens only at readout:

$$\hat{A}_{q,r} = \text{sgn}(A_{q,r}), \quad \hat{B}_{q,r} = \text{sgn}(B_{q,r}), \quad \hat{C}_{q,r} = \text{sgn}(C_{q,r}).$$

This model is closest in spirit to bond- or channel-based cellular systems such as hexagonal lattice-gas automata and to graph-based generalizations of cellular automata, where locality and bounded-degree causality are fundamental. [\[8\]](#)

Sample visual output

A typical **settled motif grid** on an open patch can be rendered by writing each cell as its six-edge motif string:

```

000111 100011 011100 101010 000111
011100 101010 010101 110001 001110
110001 000111 101010 011100 100011
111000 011100 100011 010101 111000

```

Read this as: the visible “objects” are not independently chosen cells. They are repeated local closures generated by alternating rail families.

Propagation, settling, and emergent genlock

The discrete-time CA description is useful, but real hardware does not jump from one fully legal state to another. It propagates, shares charge, conflicts, settles, and only then becomes binary. This is the right place to formalize the user’s “genlock” idea.

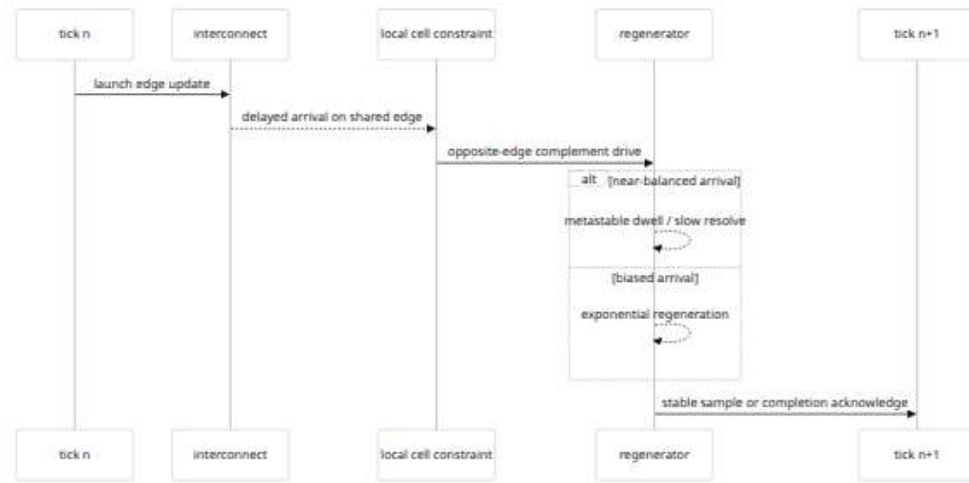
Propagation pipeline



This is not just a conceptual loop. At the circuit level, interconnect delay, differential imbalance, and regenerative feedback really do create a two-stage process: **first analog propagation, then discrete readout**. RC delay remains fundamental for wires and long bitlines, while cross-coupled or comparator stages turn a small differential into a rail-to-rail decision. Commodity DRAM latency is dominated largely by long-bitline capacitance and sense-structure tradeoffs; asynchronous dual-rail circuits make completion explicit rather than hiding it under a clock. [9]

Continuous time between clock ticks

A useful timeline is:



What matters here is that there is a **causal cone**. On a local graph CA, influence cannot outrun locality; on a physical lattice, influence cannot outrun propagation and settling. This is why the user’s “genlock is emergent” statement is technically sound: if one local relation changes, its neighbors must update, and that reconciliation wave cannot propagate faster than the medium allows. Graph-based CA theory formalizes locality; asynchronous circuit design formalizes causal completion without a global phase reference. [10]

Metastability and the seam

The Nexus “seam” has a standard circuit counterpart: the unstable midpoint of a bistable system. In a cross-coupled latch or dynamic comparator, if the competing drives are nearly balanced, regeneration is slow; if they are sufficiently unbalanced, the state resolves quickly to one attractor. Comparator literature describes the same effect operationally as degraded delay for small input differences, while metastability theory shows that no finite-time architecture can absolutely eliminate such undecidable midpoints for unsynchronized inputs. [11]

A useful local approximation for regenerative differential voltage v_d is

$$v_d(t) \approx v_d(0)e^{t/\tau_{\text{regen}}},$$

so the time to reach a safely readable logic gap V_L scales as

$$t_{\text{resolve}} \approx \tau_{\text{regen}} \ln \frac{V_L}{|v_d(0)|}.$$

Small initial difference means long dwell. In Nexus language: near the seam, subtraction has not yet collapsed into a side.

Timing hazards and the emergence of a coherence ceiling

The user's "genlock" becomes engineering timing closure when a state is observed too early. In a synchronous design, the safe observation period must exceed worst-case propagation and settling plus margin. In a compact Nexus-style expression:

$$T_{\text{sample}} \gtrsim t_{\text{wire,max}} + t_{\text{logic,max}} + t_{\text{regen,max}} + t_{\text{setup}} + t_{\text{skew/jitter}}.$$

If the sample period drops below that bound, the system begins to observe partially settled states. That is exactly where race conditions, hazards, and metastability leak into behavior. Hazard-free realizations are possible for many functions, but the literature also shows that hazard freedom can be significantly more expensive than ordinary Boolean implementation. Asynchronous dual-rail and return-to-zero protocols trade raw compactness for explicit completion and robustness. [12]

The corresponding lattice notion is:

$$v_{\text{coh}} \lesssim \frac{\ell_{\text{hop}}}{\tau_{\text{edge}} + \tau_{\text{regen}}},$$

where ℓ_{hop} is one edge hop, τ_{edge} is transport delay, and τ_{regen} is local restoration time. This is the **emergent propagation/coherence ceiling**. It is the physical version of the user's "genlock."

Hardware realizations and carrier technologies

CMOS inverter as the subtractive primitive

In standard logic, the inverter is the primitive that most directly realizes the Nexus fold. One device network pulls the output up while the complementary network pulls it down, so the output is driven toward the **opposite rail** of the input. Modern analyses still use the CMOS inverter as the canonical nonlinear, noisy decision element, and it remains the basic NOT gate on which larger logic is built. [13]

In Nexus form:

- rails: V_{DD} and ground,
- seam: the switching threshold,
- gap/state: the node voltage or differential node voltage,
- fold: $x \mapsto 1 - x$,
- closure: restored rail-level output.

This is why the user's "NOT is subtractive" claim is not a poetic extra. It is the correct first abstraction.

Cross-coupled latches, flip-flops, SRAM, and sense amplifiers

A latch is two inverters reflected into each other. The result is a bistable constraint system. Cross-coupled CMOS latches are now used not only for storage but also as physical computational primitives in latch-based Ising machines, where the hardware computes by settling into one of two minima. [14]

The same motif underlies SRAM. A conventional 6T SRAM cell stores a bit in complementary internal states Q and $\overline{Q}$. Recent SRAM in-memory computing work explicitly exploits this complementary structure, including storing complementary operand information in $Q/\overline{Q}$ and performing NAND/NOR/XOR-like operations close to or inside the array. [15]

Sense amplifiers are the dynamic readout form of the same story. The incoming signal is a small differential; the cross-coupled or comparator-like structure regenerates it into a full binary decision. Dynamic comparator design literature measures exactly this: smaller initial differentials take longer to regenerate. DRAM extends the idea further by storing one bit as capacitor charge and then using charge sharing plus sense amplification to restore a readable rail-level state, with long bitline capacitance becoming the central latency cost. [16]

Interconnect, buses, and echoes

Interconnect is where the user's rail picture becomes most literal. A bus is a repeated pattern of constrained channels. Differential buses, dual-rail asynchronous links, BL/BLB memory pairs, and parallel optical waveguide arrays are all examples of many admissible gaps moving together. Delay-insensitive asynchronous circuits go even further by explicitly encoding one bit as a pair of wires and completing transfer only when the pattern is locally valid. [17]

The "one class, many echoes" hardware interpretation is also straightforward:

- one inverter or latch cell **class** exists in the library,
- many physical **instances** or echoes are placed across the chip,
- one SRAM or photonic pSRAM bitcell class is tiled into an array,
- one logic relation becomes many local embodiments.

Hardware carrier comparison

The following table compares four carrier families for the Nexus ideas. The mapping is synthesized from current literature on CMOS logic, DRAM/SRAM, MTJ-based memories, and phase-change or interferometric photonic memory. [18]

Carrier	Physical state variable	Rails / constraints	Gap / readable state	Reflection mechanism	Strengths	Main limitations
CMOS transistor logic	node voltage / differential voltage	V_{DD} , ground, threshold margins	voltage gap or ΔV	inverter, static CMOS, dynamic latch	mature, dense, fast, well-understood	leakage, interconnect RC, metastability
Capacitor-based memory	stored charge on capacitor or dynamic node	precharge midpoint, BL/BLB symmetry,	tiny charge-sharing ΔV	sense amplifier restore and complement	very dense storage, natural differential	destructive read, refresh, bitline capacitance, disturbance

Carrier	Physical state variable	Rails / constraints	Gap / readable state	Reflection mechanism	Strengths	Main limitations
		refresh timing		t	readout	
Spintronic MTJ / MRAM	free-layer magnetization, tunnel resistance	parallel vs antiparallel magnetic state	ΔR / TMR contrast	write-pulse–induced magnetic switching, sensed by comparator	nonvolatile, good endurance, memory-compute potential	write current, variability, sensing margins
Photonic memory / logic	phase, amplitude, resonance state	interferometer arms, reference phase, PCM state	interference contrast, resonance shift	MZI phase inversion, PCM transition	high bandwidth, low transport loss, wavelength parallelism	device area, control/calibration, static tuning unless nonvolatile PCM used

The photonic row deserves a special note. There is no exact optical analogue of a DC CMOS rail pair, so the photonic “rails” are usually **two paths, two phases, two resonance states, or a reference plus a shifted state**. That still fits Nexus well: the readable state is again relational rather than absolute. Phase-change photonic memories and cross-coupled photonic SRAM proposals make this explicit by combining nonvolatile state setting with differential or interferometric readout. [19]

Collapse memory, feedforward learning, and architecture implications

At the algorithmic level, the strongest published support for the user’s “memory by collapse” view comes from **associative attractor systems**. Classical Hopfield networks store patterns in a symmetric energy landscape and retrieve by settling from partial or corrupted cues; modern or dense Hopfield models sharpen the landscape and raise usable capacity; and modern Hopfield theory explicitly includes global minima, subset-averaging metastable states, and single-pattern fixed points. Ramsauer et al. then show that the modern Hopfield update is mathematically equivalent to transformer attention, which means the collapse primitive already exists inside contemporary deep learning. [20]

That matters because it reframes the architecture question. If attention is already a collapse step, then a Nexus architecture is not “anti-transformer” in the trivial sense. It is better described as **collapse-forward rather than expansion-forward**: make the settling operation primary and make global feedforward expansion secondary or optional.

The user-provided **Memory by Collapse** paper pushes that argument from theory into engineering comparison. Under its toy-scale continual-write protocol, the dense cell writes by append or local fold, recalls by settling from fragments, preserves earliest items after 1,000 one-shot writes, and reports much lower write cost than the parameter-matched gradient-trained baseline in the tested setup. It also places a

useful negative beside the positive: the classical cell hits the familiar Hopfield capacity wall, while the sharpened cell removes that wall under the tested conditions. Those are exactly the kinds of measurements one would want if testing whether “collapse cells” can outperform global weight rewriting in settings that stress local continual retention. [20]

Feedforward learning, by contrast, has the well-known problem that sequential learning tends to overwrite shared weights unless one adds rehearsal, consolidation penalties, architectural separation, or other countermeasures. Elastic Weight Consolidation is one example of a mitigation strategy; the uploaded Nexus comparison uses a different tactic, namely to shift the burden from global weight drift to local settling and append-style memory. That is not a proof that global gradient methods should disappear, but it is a precise architectural hypothesis: **local collapse cells should compose into higher-level collapse hierarchies** more naturally than endlessly widening feedforward stacks. [21]

A cautious architectural implication is therefore:

- use feedforward expansions for feature extraction when necessary,
- use local collapse cells for memory, arbitration, retrieval, and symbolic closure,
- organize memories hierarchically so that higher cells settle over the stable readouts of lower cells,
- allow the clock or handshake schedule to emerge from measured settle times rather than imposing optimism from above.

That last point is especially congruent with the user’s genlock idea. The architecture should not sample until the collapse is done.

Failure modes, experiments, and source priorities

Failure modes

The relevant failure modes are now easy to classify in Nexus language.

Noise perturbs the gap variable. In CMOS it narrows the usable voltage gap; in DRAM it distorts tiny charge-sharing differentials; in MTJs it corrupts resistance contrast; in photonics it detunes phase or resonance. The countermeasure is always the same: stronger constraints, larger margins, or regenerative restoration. [22]

Metastability occurs when the seam is sampled before it has chosen a side. This is the direct failure of closure timing. Cross-coupled and comparator structures can only reduce its probability or give it more time to resolve, not abolish it in finite time for unsynchronized arrivals. [11]

Race conditions and hazards occur when logically equivalent constraint paths arrive at different times, briefly exposing illegal intermediate states. Hazard-free circuits exist, but the cost can be materially higher than ordinary Boolean implementations. Delay-insensitive encodings reduce the hidden assumptions by making validity and completion explicit. [12]

Thermal and erasure costs attach most strongly to irreversible collapse that destroys alternative possibilities rather than to reversible propagation itself. Landauer’s principle and modern experiments near the Landauer bound are the right frame here: the thermodynamic bill is paid when information is irreversibly thrown away. [23]

Dense-array disturbance shows what happens when constraint isolation is too weak. RowPress and ColumnDisturb demonstrate that neighboring memory rows and columns can corrupt one another when activation and settling assumptions are overrun. This is the hardware version of a lattice losing local containment. [24]

The proposed hex model prevents global collapse precisely by localizing inconsistency. Opposite-edge complement keeps each cell balanced; shared-edge consistency forces interface agreement; bounded-degree locality limits causal spread; and completion-aware sampling prevents observation of unfinished states. None of these makes the system invulnerable. They only prevent arbitrary drift.

Experiment matrix

The most direct way to test the Nexus-to-hardware mapping is to simulate both the abstract lattice and the hardware kernels that instantiate it. A practical platform is **Python + NumPy/Numba** for the lattice engine, plus a simple **event-driven simulator** for asynchronous edge arrivals and regenerative settle events. Small circuit kernels can then be cross-checked in SPICE or Verilog-A.

Experiment	Goal	Configuration	Key parameters	Initial condition	Metrics	Expected visualization
Noise to closure	measure how fast random edge noise settles to legal motifs	open hex patch 64×64	$J_A = J_B = J_C = 1$, $\beta = 3,5,8$, noise $\sigma = 0 \dots 0.3$, $dt = 0.01\tau$	random analog A, B, C , fixed legal boundary	settle time, final mismatch density, motif entropy	heatmap of illegal-edge count vs time
Boundary cue retrieval	test memory-as-collapse on the lattice	open patch 128×128	cue clamp fraction 10% ... 50%, noise 0 ... 0.2	legal stored boundary, corrupted interior	recall accuracy, basin depth, coherence length	before/after motif grids, cue-to-closure movie
Single-edge impulse	measure propagation cone and speed	strip 256×32	$\tau_{\text{edge}} = 1$, $\tau_{\text{regen}} = 0.1, 0.5, 1.0$	one flipped edge at center	front speed, attenuation, cone width	space-time raster along rail family
Collision and seam dwell	quantify metastable conditions	patch 64×64 with dual wavefronts	arrival skew $\Delta t = -1 \dots +1$, noise $10^{-6} \dots 10^{-2}$	two opposing boundary updates	dwell-time distribution, wrong-settle rate	histogram of dwell times, seam heatmap
Genlock sweep	find coherence ceiling	same patch, sampled synchronous	$T_{\text{sample}}/\tau_{\text{settle}} = 0.2 \dots 5$	repeated random cues	error rate, hazard count, max coherent	BER vs sample period,

Experiment	Goal	Configuration	Key parameters	Initial condition	Metrics	Expected visualization
	under sampling	y			sample rate	phase diagram
Hardware kernel co-sim	tie lattice events to real circuit primitives	inverter, cross-coupled latch, sense amp	PVT corners, input skew, bitline C, ΔV seed	pulse, small differential, asynchronous arrivals	resolve time, energy/decision, metastability tail	waveform overlays, resolve-time scatter
Carrier substitution	compare CMOS, capacitor, MTJ, photonic realizations	same abstract rule, different settle kernels	voltage, charge, TMR, phase/resonance noise models	matched cue patterns	energy proxy, delay, error rate	carrier comparison radar or line plots
Hierarchical collapse	test cells-over-cells architecture	4-level lattice pyramid	lower-level settle threshold, upper-level cue density	correlated motif blocks	recall under partial cues, capacity scaling	multiscale closure animation

Concrete simulation parameters

A good first sweep is:

- lattice sizes: 32×32 , 64×64 , 128×128 ,
- open boundaries first, toroidal boundaries second,
- couplings $J_A = J_B = J_C = 1$,
- saturation $\beta \in \{3, 5, 8\}$,
- noise standard deviation $\sigma \in [0, 0.2]$,
- event-driven delays $\tau_{\text{edge}} \in [0.5, 2.0]$, $\tau_{\text{regen}} \in [0.1, 1.0]$,
- sampling periods from $0.2\tau_{\text{settle}}$ to $5\tau_{\text{settle}}$,
- 100–1,000 random seeds per condition for error bars.

The **minimum useful data products** are:

- motif-ID snapshots,
- illegal-edge heatmaps,
- mismatch density over time,
- propagation space-time rasters,
- histograms of metastable dwell,
- BER versus sampling period,

- energy proxy versus settle quality.

Sample visualization expectations

A typical progression should look like this:

Frame 0 : random edge field, high mismatch density
Frame 5 : alternating rail families begin to emerge
Frame 12 : most cells are legal, only seam collisions remain
Frame 20 : complete closure into eight legal motif classes

And the animation most worth making is not a cell-fill animation but an **edge-state animation**, where illegal shared edges are highlighted and the viewer watches the conflict front shrink.

Recommended next steps

The immediate next step is to implement the lattice in the **three-field parameterization** (A, B, C) rather than on raw six-edge cells. That reduces memory, makes the rail families explicit, and directly exposes the boundary-seed interpretation.

The second next step is to build a **two-timescale simulator**:

- fast event-driven edge propagation,
- slower regenerative settle and observation.

The third is to couple that simulator to real hardware kernels:

- CMOS inverter,
- cross-coupled latch,
- differential sense amplifier,
- one non-CMOS carrier such as MTJ or photonic PCM.

The fourth is to test the user's memory hypothesis directly: use the same settle engine both for lattice closure and for associative recall, so that the "edge-first universe" and the "memory-by-collapse" engine share one computational core.

Primary sources to prioritize

If the goal is to turn Nexus into a publishable technical program, the best reading order is:

- **Hopfield 1982**, for classical attractor memory and energy descent. [\[25\]](#)
- **Amit, Gutfreund, Sompolinsky 1985**, for the classical capacity wall that the uploaded Nexus paper also foregrounds. The user's uploaded file summarizes this role clearly. [\[filecite@turnofile1\]](#)
- **Krotov and Hopfield 2016**, for dense associative memory. [\[26\]](#)
- **Ramsauer et al. 2020**, for the attention = modern Hopfield update identity. [\[27\]](#)

- **Chaney and Molnar 1973** and **Ginosar's metastability tutorial**, for seam behavior, synchronizers, and why finite-time perfect arbitration is impossible. A concise web-accessible summary cites both. [28]
- **Elmore 1948**, for the propagation side of genlock and why wires set ceilings. A summary page is enough for planning, but the original paper should be read directly in implementation work. [29]
- **Hazard-free and delay-insensitive literature**, especially on dual-rail and return-to-zero protocols. [30]
- **Hexagonal/graph CA literature**, for locality, bounded-degree causality, and non-square automata design space. [8]
- **SRAM/DRAM compute and sense papers**, for memory as differential regeneration rather than mere storage. [31]
- **Spintronic and photonic memory papers**, for non-CMOS carrier realizations of rails, gaps, and collapse. [32]
- **The user's uploaded Memory by Collapse material**, because it already contains the most Nexus-specific experimental claims and the architecture hypothesis this report is mapping outward into hardware. [filecite?turnofileo?filecite?turnofile1?](#)

The most concise verdict is this: the Nexus framework is not yet a theorem of computation, but it is already a **technically meaningful constraint vocabulary**. NOT really is subtractive. Latches and sense amps really are collapse cells. Differential buses and bitlines really do make state live in gaps between rails. The proposed hex edge automaton really does move freedom from area to boundary. And genlock really can be read as an emergent propagation ceiling rather than an arbitrary imposed tempo. The next decisive test is not philosophical. It is whether a shared settle engine can power both the abstract lattice and a useful associative hardware-memory stack.

[1] [4] [13] [18] [22] <https://arxiv.org/abs/2205.12659>

<https://arxiv.org/abs/2205.12659>

[2] [8] [10] <https://arxiv.org/abs/1212.0027>

<https://arxiv.org/abs/1212.0027>

[3] [20] [27] <https://arxiv.org/abs/2008.02217>

<https://arxiv.org/abs/2008.02217>

[5] [11] [16] <https://arxiv.org/abs/2112.05924>

<https://arxiv.org/abs/2112.05924>

[6] <https://arxiv.org/abs/2503.19544>

<https://arxiv.org/abs/2503.19544>

[7] [17] Asynchronous Early Output Dual-Bit Full Adders Based on Homogeneous and Heterogeneous Delay-Insensitive Data Encoding

https://arxiv.org/abs/1704.07619?utm_source=chatgpt.com

[9] <https://arxiv.org/abs/1805.03048>

<https://arxiv.org/abs/1805.03048>

[12] [30] <https://arxiv.org/abs/1711.01904>

<https://arxiv.org/abs/1711.01904>

[14] <https://arxiv.org/abs/2205.14729>

<https://arxiv.org/abs/2205.14729>

[15] [31] <https://arxiv.org/abs/2008.03378>

<https://arxiv.org/abs/2008.03378>

[19] <https://arxiv.org/abs/1208.1417>

<https://arxiv.org/abs/1208.1417>

[21] <https://arxiv.org/abs/1612.00796>

<https://arxiv.org/abs/1612.00796>

[23] <https://arxiv.org/abs/1411.6730>

<https://arxiv.org/abs/1411.6730>

[24] <https://arxiv.org/abs/2306.17061>

<https://arxiv.org/abs/2306.17061>

[25] Hopfield network

https://en.wikipedia.org/wiki/Hopfield_network?utm_source=chatgpt.com

[26] <https://arxiv.org/abs/1606.01164>

<https://arxiv.org/abs/1606.01164>

[28] https://en.wikipedia.org/wiki/Metastability_%28electronics%29

https://en.wikipedia.org/wiki/Metastability_%28electronics%29

[29] https://en.wikipedia.org/wiki/Elmore_delay

https://en.wikipedia.org/wiki/Elmore_delay

[32] <https://arxiv.org/abs/0803.2254>

<https://arxiv.org/abs/0803.2254>